

Backend Engineering Handbook

A practical reference on databases, distributed systems,
and modern AI infrastructure

Version 1.0 • Internal Reference Document

1. Database Fundamentals

1.1 Indexing

An index is a data structure that improves the speed of data retrieval operations on a database table at the cost of additional writes and storage space. Without an index, a database must perform a full collection scan, examining every document to find matches. With a B-tree index, the database can locate matching documents in logarithmic time. Indexes should be created on fields that appear frequently in query filters and sort operations. However, over-indexing degrades write performance because every insert and update must also update all relevant indexes.

1.2 Replication

Replication maintains multiple copies of data across different servers to provide high availability and fault tolerance. In a replica set, one node acts as the primary and receives all write operations, while secondary nodes replicate the primary's data. If the primary fails, an automatic election promotes a secondary to primary, ensuring the system remains available. Replication also enables read scaling by directing read queries to secondary nodes, though this can introduce stale reads due to replication lag.

1.3 Sharding

Sharding horizontally partitions data across multiple machines to support datasets and throughput that exceed the capacity of a single server. A shard key determines how documents are distributed across shards. Choosing a good shard key is critical: a poorly chosen key can lead to uneven data distribution, creating hotspots where one shard handles a disproportionate share of the load. Common strategies include hashed sharding for even distribution and ranged sharding for efficient range queries.

2. Distributed Systems Patterns

2.1 The Saga Pattern

The Saga pattern manages data consistency across microservices in distributed transaction scenarios. Rather than using a single ACID transaction spanning multiple services, a saga breaks the transaction into a sequence of local transactions, each updating a single service. If one step fails, the saga executes compensating transactions to undo the preceding steps. Sagas can be coordinated through orchestration, where a central coordinator directs each step, or choreography, where services react to events published by other services.

2.2 Circuit Breaker

The circuit breaker pattern prevents an application from repeatedly trying to execute an operation that is likely to fail. When failures reach a threshold, the circuit breaker trips and subsequent calls fail immediately without attempting the operation, giving the failing service time to recover. After a timeout, the breaker enters a half-open state and allows a limited number of test requests through. If those succeed, the breaker closes and normal operation resumes.

2.3 CQRS

Command Query Responsibility Segregation separates read and write operations into distinct models. The command model handles creates, updates, and deletes, while the query model is optimized purely for reads. This separation allows each side to scale independently and use data representations best suited to its purpose. CQRS is often paired with event sourcing, where state changes are persisted as an immutable log of events rather than overwriting current state.

3. AI Infrastructure

3.1 Vector Embeddings

A vector embedding is a numerical representation of data such as text, images, or audio, expressed as a fixed-length array of floating point numbers. Embeddings capture semantic meaning: items with similar meaning are positioned close together in the high-dimensional vector space. Embeddings are produced by neural network models trained on large datasets. For text, sentence transformer models convert a passage into a dense vector that can be compared against others using similarity metrics.

3.2 Approximate Nearest Neighbor Search

Finding the exact closest vectors to a query across millions of records is computationally expensive because it requires comparing the query against every stored vector. Approximate nearest neighbor algorithms trade a small amount of accuracy for dramatic speed improvements. HNSW, the most widely used ANN algorithm, organizes vectors into a multi-layer graph where higher layers enable long-range jumps and lower layers provide fine-grained local search. This structure allows sub-millisecond search across very large vector collections.

3.3 Retrieval Augmented Generation

Retrieval augmented generation, or RAG, enhances large language models by grounding their responses in an external knowledge base. When a user asks a question, the system first embeds the question and performs a vector search to retrieve the most relevant documents. These documents are then inserted into the prompt as context, and the language model generates an answer based on the retrieved information rather than relying solely on its training data. This approach reduces hallucination and allows the model to answer questions about private or recent information it was never trained on.

3.4 Chunking Strategies

Before documents can be embedded and stored, long texts must be split into smaller chunks. Chunking is important because embedding models have input length limits, and smaller chunks produce more precise retrieval. A common strategy is fixed-size chunking with overlap, where each chunk contains a set number of characters or tokens and consecutive chunks share a portion of text to preserve context across boundaries. More advanced strategies split on semantic boundaries such as paragraphs or sentences.